

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



BİL-304 **BİLGİSAYAR MİMARİSİ** **Dr. Öğr. Üyesi Fatih GÖKÇE**

10. Hafta

Bölüm / Hafta Hedefleri

- **Ardışıl İşlemci (SEQ) mimarisinin 1. Kısımını anlatacağımız bu derste**
 - **Ardışıl Y86-64 işlemcisinin aşamaları,**
 - **Y86-64 komut setindeki komutların hesaplama aşamalarının detayları tanıtılacaktır.**

Bu ders notundaki slaytlar ders kitabına ait sunum dosyalarından adapte edilmiştir:
<http://csapp.cs.cmu.edu/>

Adapted from slides of the textbook: <http://csapp.cs.cmu.edu/>

SEQ Aşamaları

Çekme (Fetch)

- Komut hafızasından komut okunur

Çözme (Decode)

- Program kaydedicileri okunur

Yürütme (Execute)

- Değer veya adres hesaplanır

Hafıza (Memory)

- Veri okunur veya yazılır

Geri Yazma (Write Back)

- Program kaydedicilerdeki güncellenmesi gereken değerler yazılır

PC

- Program Sayacı güncellenir

Slaytların devamında SEQ Aşamalarının İngilizceleri Kullanılacaktır!

PC

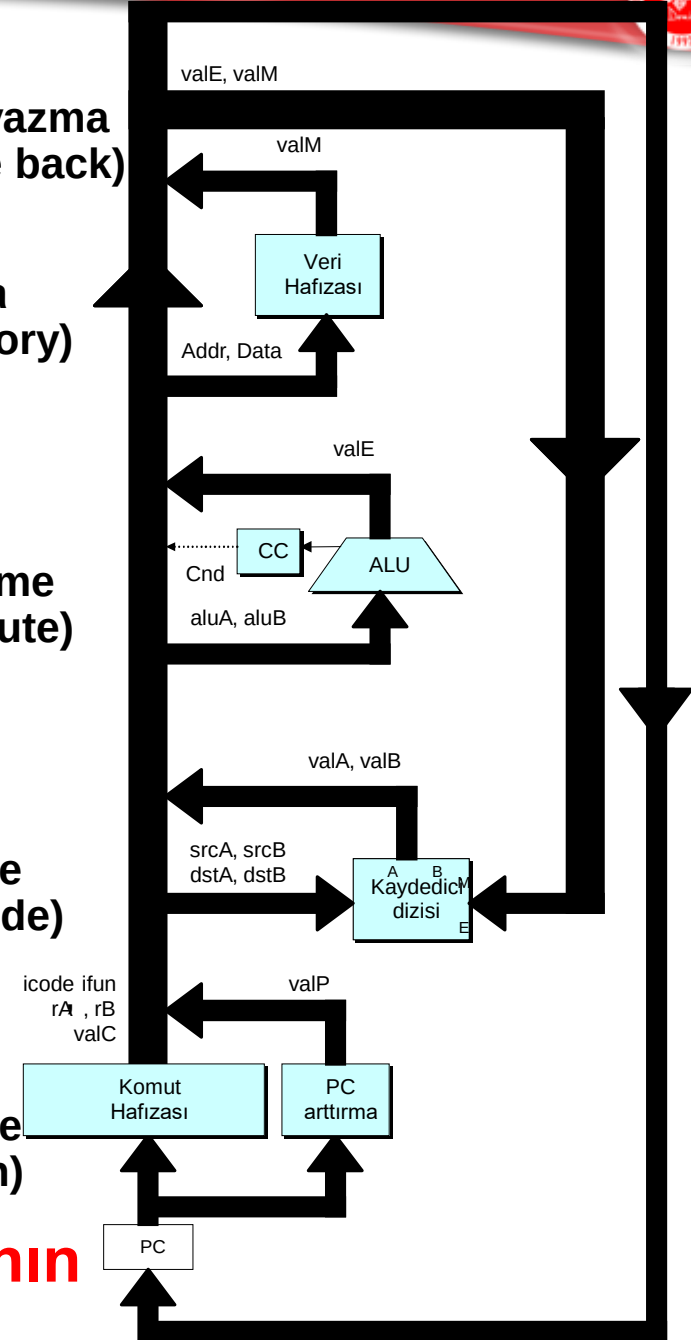
Geri yazma
(Write back)

Hafıza
(Memory)

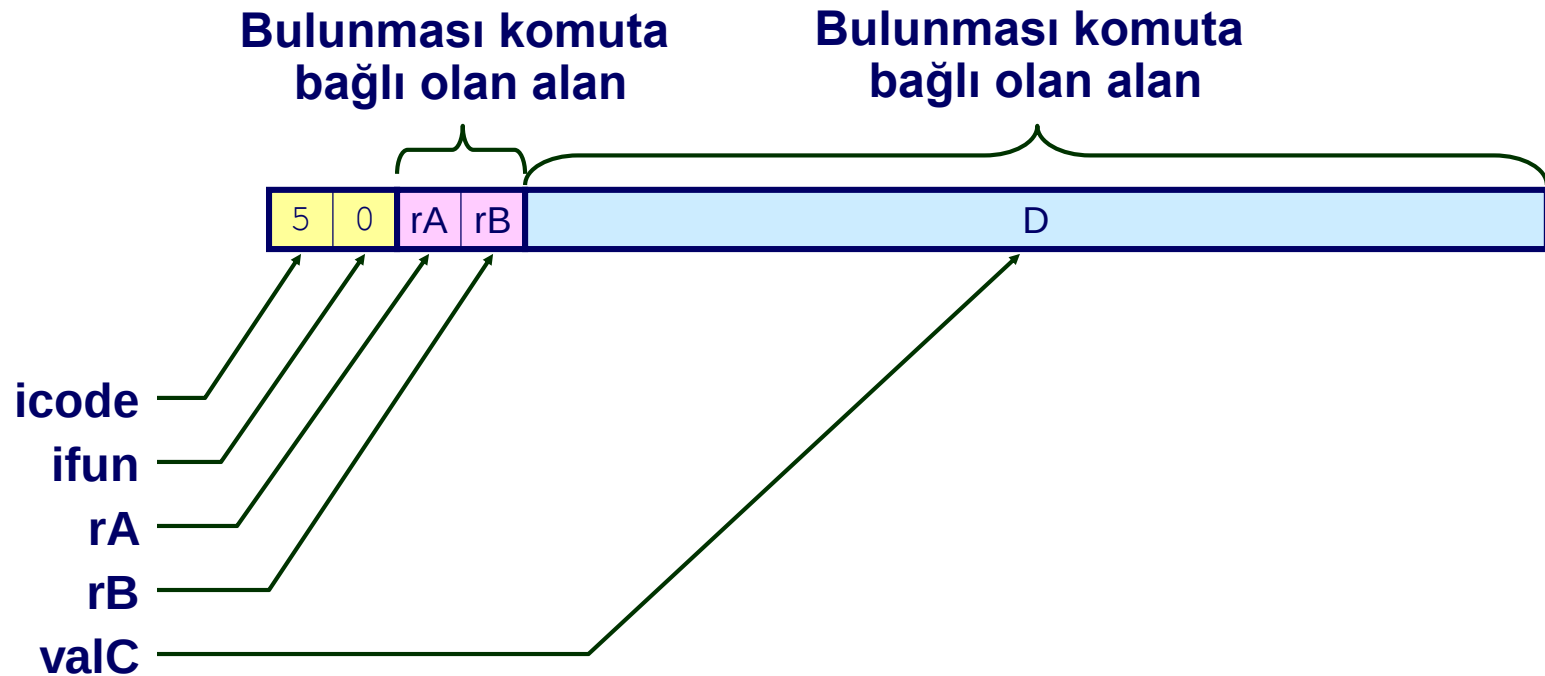
Yürütme
(Execute)

Çözme
(Decode)

Çekme
(Fetch)



Komutların Çözülmesi



Komut Formatı

- Komut byte'ı
 - Bulunması komuta bağlı olan kaydedici byte'ı
 - Bulunması komuta bağlı olan sabit sözcük
- icode:ifun
rA:rB
valC

Aritmetik/Lojik İşlemlerin Yürütülmesi

OPq rA, rB

6	fn	rA	rB
---	----	----	----

Fetch

- 2 byte okunur

Decode

- İşlem yapılacak kaydediciler okunur

Execute

- İşlem gerçekleştirilir
- Durum kodları (condition codes) güncellenir

Memory

- Hiçbir şey yapılmaz

Write back

- Kaydediciler güncellenir

PC Update

- PC değeri 2 arttırılır

Hesaplama Aşamaları: Aritmetik/Lojik İşlemler

	OPq rA, rB	
Fetch	$icode:ifun \leftarrow M_1[PC]$ $rA:rB \leftarrow M_1[PC+1]$ $valP \leftarrow PC+2$	Komut byte'ını oku Kaydedici byte'ını oku Bir sonraki PC'yi hesapla
Decode	$valA \leftarrow R[rA]$ $valB \leftarrow R[rB]$	İşlem yapılacak A değerini oku İşlem yapılacak B değerini oku
Execute	$valE \leftarrow valB \text{ OP } valA$ Set CC	ALU işlemini gerçekleştir Durum kodu kaydedicisini güncelle
Memory		
Write back	$R[rB] \leftarrow valE$	Sonucu geri yaz
PC update	$PC \leftarrow valP$	PC'yi güncelle

- Komutun yürütülmesi basit adımların dizisi olarak ifade edilir
- Tüm komutlar için aynı genel form kullanılır

rmmovq'nin yürütülmesi

rmmovq rA, D (rB)

4	0	rA	rB
---	---	----	----

D

Fetch

- 10 byte okunur

Decode

- İşlem yapılacak kaydediciler okunur

Execute

- Kaydedici değerinin yazılacağı efektif adres hesaplanır

Memory

- Hafızaya yazma işlemi yapılır

Write back

- Hiçbir şey yapılmaz

PC Update

- PC değeri 10 arttırılır

Hesaplama Aşamaları: `rmmovq`

	<code>rmmovq rA, D(rB)</code>
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC}+1]$ $\text{valC} \leftarrow M_8[\text{PC}+2]$ $\text{valP} \leftarrow \text{PC}+10$
Decode	$\text{valA} \leftarrow R[\text{rA}]$ $\text{valB} \leftarrow R[\text{rB}]$
Execute	$\text{valE} \leftarrow \text{valB} + \text{valC}$
Memory	$M_8[\text{valE}] \leftarrow \text{valA}$
Write back	
PC update	$\text{PC} \leftarrow \text{valP}$

Komut byte'ını oku

Kaydedici byte'ını oku

D ofset değerini oku

Bir sonraki PC'yi hesapla

İşlem yapılacak A değerini oku

İşlem yapılacak B değerini oku

Kaydedici değerinin yazılacağı
efektif adresi hesapla

Değeri hafızaya yaz

PC'yi güncelle

- Efektif adres hesaplaması için ALU kullanılır

popq'nun yürütülmesi

popq rA

b	0	rA	F
---	---	----	---

Fetch

- 2 byte okunur

Decode

- Yığın pointer'i okunur

Execute

- Yığın pointer'ini 8 arttır

Memory

- Eski (8 arttırılmamış) yığın pointer'inin gösterdiği hafıza adresinden oku

Write back

- Yığın pointer'ini güncelle
- Hafızadan okunan değeri kaydediciye yaz

PC Update

- PC'yi 2 arttır

Hesaplama Aşamaları: popq

	popq rA	
Fetch	icode:ifun $\leftarrow M_1[PC]$ rA:rB $\leftarrow M_1[PC+1]$	Komut byte'ını oku Kaydedici byte'nı oku
	valP $\leftarrow PC+2$	Bir sonraki PC'yi hesapla
Decode	valA $\leftarrow R[\%rsp]$ valB $\leftarrow R[\%rsp]$	Yığın pointer'ini oku Yığın pointer'ini oku
	valE $\leftarrow valB + 8$	Yığın pointer'ini arttır
Memory	valM $\leftarrow M_8[valA]$	Yığından oku
Write back	R[%rsp] $\leftarrow valE$ R[rA] $\leftarrow valM$	Yığın pointer'ini güncelle Sonucu geri yaz
	PC update	PC'yi güncelle

- Yığın pointer'ini arttırmak için ALU kullanılır
- 2 kaydedicinin güncellenmesi gerekir
 - Yığından çekilen değerin yazılacağı kaydedici
 - Yeni yığın pointer değerinin yazılacağı kaydedici

Şartlı Taşıma Komutlarının Yürütülmesi

`cmovXX rA, rB`

2	fn	rA	rB
---	----	----	----

Fetch

- 2 byte okunur

Decode

- İşlem yapılacak kaydediciler okunur

Execute

- Eğer `cnd=0` ise, hedef kaydedici 0xF yapılır

Memory

- Hiçbir şey yapılmaz

Write back

- Kaydedici güncellenir (hedef 0xF yapılmışsa rB ile komutta verilen kaydedici güncellenmemiş olur.)

PC Update

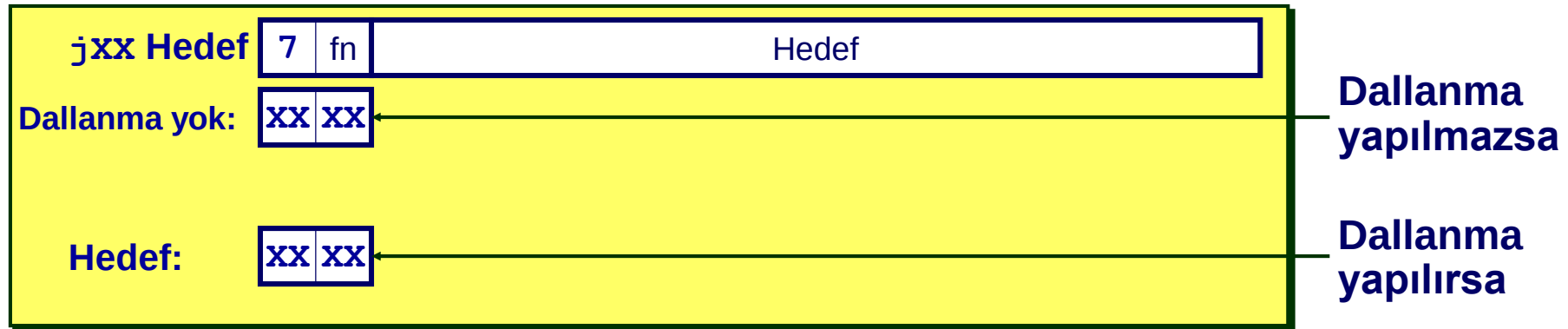
- PC'yi 2 arttır

Hesaplama Aşamaları: Şartlı Taşıma

	cmovXX rA, rB	
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC}+1]$ $\text{valP} \leftarrow \text{PC}+2$	Komut byte'ını oku Kaydedici byte'nı oku Bir sonraki PC'yi hesapla
Decode	$\text{valA} \leftarrow R[\text{rA}]$ $\text{valB} \leftarrow 0$	İşlem yapılacak A değerini oku
Execute	$\text{valE} \leftarrow \text{valB} + \text{valA}$ If ! Cond(CC,ifun) rB $\leftarrow 0xF$	valA 'yı ALU üzerinden aktar (Şart sağlanmamışsa kaydedici güncellemesini engelle)
Memory		
Write back	$R[\text{rB}] \leftarrow \text{valE}$	Sonucu geri yaz
PC update	$\text{PC} \leftarrow \text{valP}$	PC'yi güncelle

- rA kaydedicisi okunur ve değeri ALU üzerinden aktarılır ($\text{valB} = 0$, etkisiz eleman olduğu için)
- Hedef kaydedicinin rB yerine 0xF yapılması taşımayı iptal etmek demektir
 - Eğer durum kodları ve taşıma şartına bakıldığında şart sağlanmayıp taşıma yapılmaması gerekiyorsa

Dallanmaların (Jumps) Yürütülmesi



Fetch

- 9 byte okunur
- PC 9 arttırılır

Decode

- Hiçbir şey yapılmaz

Execute

- Dallanma şartı ve durum kodlarına bağlı olarak dallanma yapılıp yapılmayacağı belirlenir

Memory

- Hiçbir şey yapılmaz

Write back

- Hiçbir şey yapılmaz

PC Update

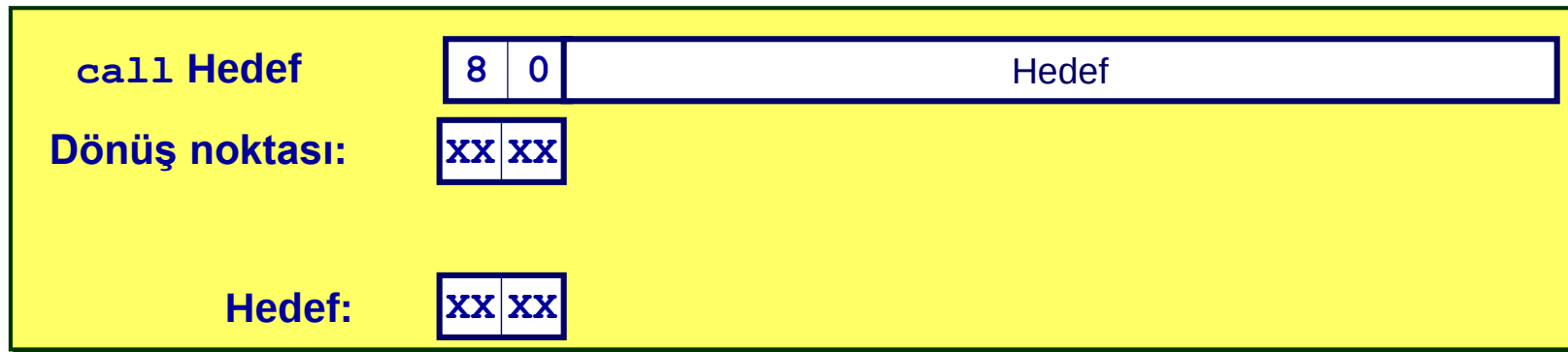
- PC'ye dallanma yapılırsa Hedef adres değeri yüklenir, yapılmazsa arttırılmış PC değeri yüklenir

Hesaplama Aşamaları: Dallanmalar

	jXX Hedef	
Fetch	$\text{icode:ifun} \leftarrow M_1[PC]$ $\text{valC} \leftarrow M_8[PC+1]$ $\text{valP} \leftarrow PC+9$	Komut byte'ını oku Hedef adres değerini oku Dallanma yapılmazsa işletilecek komutun adresi
Decode		
Execute	$\text{Cnd} \leftarrow \text{Cond}(\text{CC}, \text{ifun})$	Dallanma yapılacak mı?
Memory		
Write back		
PC update	$\text{PC} \leftarrow \text{Cnd} ? \text{valC} : \text{valP}$	PC'yi güncelle

- Her iki adres değeri de hesaplanır
- Durum kodları ve dallanma şartına bağlı olarak iki adresten seçilmesi gereken seçilir

call'un Yürütülmesi



Fetch

- 9 byte okunur
- PC 9 arttırılır

Decode

- Yığın pointer'ı okunur

Execute

- Yığın pointer'ı 8 azaltılır

Memory

- Arttırılmış PC değeri (dönüş noktasının adresi) yığın pointer'ının yeni değeriyle gösterilen adrese yazılır

Write back

- Yığın pointer'ı güncellenir

PC Update

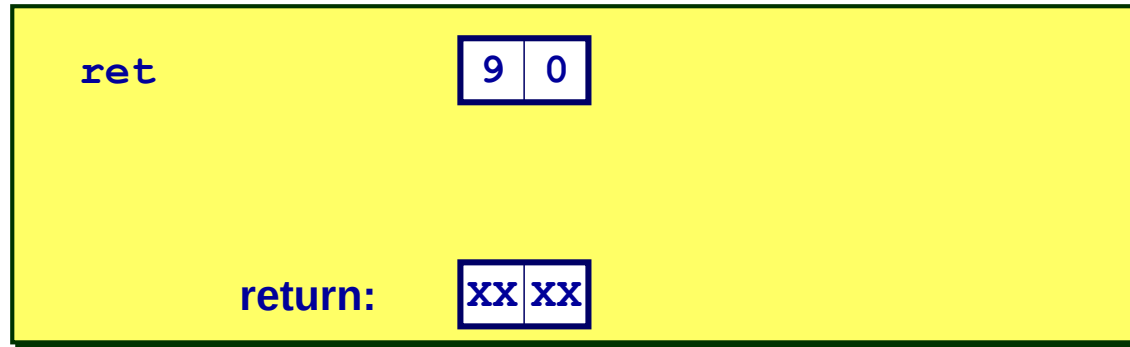
- PC'ye Hedef'in adresi yüklenir

Hesaplama Aşamaları: call

	call Hedef	
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$	Komut byte'ını oku
	$\text{valC} \leftarrow M_8[\text{PC}+1]$	Hedef adresini oku
	$\text{valP} \leftarrow \text{PC}+9$	Dönüş noktasının adresini hesapla
Decode	$\text{valB} \leftarrow R[\%rsp]$	Yığın pointer'ını oku
Execute	$\text{valE} \leftarrow \text{valB} + -8$	Yığın pointer'ını azalt
Memory	$M_8[\text{valE}] \leftarrow \text{valP}$	Dönüş noktasının adresini yığına yaz
Write back	$R[\%rsp] \leftarrow \text{valE}$	Yığın pointer'ını güncelle
PC update	$\text{PC} \leftarrow \text{valC}$	PC'ye Hedef adresini yükle

- Yığın pointer'ını azaltmak için ALU kullanılır
- Arttırılmış PC değeri (geri dönerken kullanılmak üzere yığına) kaydedilir

ret'in Yürütülmesi



Fetch

- 1 byte okunur

Decode

- Yığın pointer'ı okunur

Execute

- Yığın pointer'ı 8 arttırılır

Memory

- Geri dönüş adresi arttırılmamış yığın pointer'ının gösterdiği adresten okunur

Write back

- Yığın pointer'ını güncelle

PC Update

- PC'ye okunan geri dönüş adresini yükle

Hesaplama Aşamaları: ret

ret		
Fetch	icode:ifun $\leftarrow M_1[PC]$	Komut byte'ını oku
Decode	valA $\leftarrow R[\%rsp]$ valB $\leftarrow R[\%rsp]$	İşlem yapılacak yığın pointer'ını oku İşlem yapılacak yığın pointer'ını oku
Execute	valE $\leftarrow valB + 8$	Yığın pointer'ını arttır
Memory	valM $\leftarrow M_8[valA]$	Geri dönüş adresini oku
Write back	$R[\%rsp] \leftarrow valE$	Yığın pointer'ını güncelle
PC update	$PC \leftarrow valM$	PC'ye geri dönüş adresini yükle

- Yığın pointer'ını arttırmak için ALU kullanılır
- Geri dönüş adresi hafızadan okunur

Hesaplama Adımları

		OPq rA, rB
Fetch	icode,ifun	icode:ifun $\leftarrow M_1[PC]$
	rA,rB	rA:rB $\leftarrow M_1[PC+1]$
	valC	
	valP	valP $\leftarrow PC+2$
Decode	valA, srcA	valA $\leftarrow R[rA]$
	valB, srcB	valB $\leftarrow R[rB]$
Execute	valE	valE $\leftarrow valB \text{ OP } valA$
	Cond code	Set CC
Memory	valM	
Write back	dstE	R[rB] $\leftarrow valE$
	dstM	
PC update	PC	PC $\leftarrow valP$

Komut byte'ını oku
 Kaydedici byte'nı oku
 [Sabit sözcüğü oku]
 Bir sonraki PC'yi hesapla
 İşlem yapılacak A değerini oku
 İşlem yapılacak B değerini oku
 ALU işlemini gerçekleştir
 Durum kodlarını güncelle
 [Hafızadan oku/hafızaya yaz]
 ALU sonucunu geri yaz
 [Hafıza sonucunu geri yaz]
 PC'yi güncelle

- Tüm komutlar aynı genel biçimi izler
- Her bir adımda hesaplananlar farklılaşır

Hesaplama Adımları

		call Dest
Fetch	icode,ifun	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{valC} \leftarrow M_8[\text{PC}+1]$ $\text{valP} \leftarrow \text{PC}+9$
	rA,rB	
	valC	
	valP	
Decode	valA, srcA	$\text{valB} \leftarrow R[\%rsp]$
	valB, srcB	
Execute	valE	$\text{valE} \leftarrow \text{valB} + -8$
	Cond code	
Memory	valM	$M_8[\text{valE}] \leftarrow \text{valP}$
Write back	dstE	$R[\%rsp] \leftarrow \text{valE}$
	dstM	
PC update	PC	$\text{PC} \leftarrow \text{valC}$

Komut byte'ını oku
 [Kaydedici byte'nı oku]
 Sabit sözcüğü oku
 Bir sonraki PC'yi hesapla
 [İşlem yapılacak A değerini oku]
 İşlem yapılacak B değerini oku
 ALU işlemini gerçekleştir
 [Durum kodlarını güncelle/kullan]
 Hafızaya yaz
 ALU sonucunu geri yaz
 [Hafıza sonucunu geri yaz]
 PC'yi güncelle

- Tüm komutlar aynı genel biçimi izler
- Her bir adımda hesaplananlar farklılaşır

Hesaplanan Değerler

Fetch

icode	Komut kodu
ifun	Komut fonksiyonu
rA	Komut kaydedicisi A
rB	Komut kaydedicisi B
valC	Komut sabiti
valP	Arttırılmış PC

Decode

srcA	Kaydedici ID'si A
srcB	Kaydedici ID'si B
dstE	Hedef kaydedici E
dstM	Hedef kaydedici M
valA	Kaydedici değeri A
valB	Kaydedici değeri B

Execute

■ valE	ALU sonucu
■ Cnd	Şartlı Dallanma/taşıma bayrağı

Memory

■ valM	Hafızadan okunan veri
--------	-----------------------

- 



Kaynaklar

- 1. Computer Systems: A Programmer's Perspective, Randal E. Bryant ve David R. O'Hallaron, PEARSON.**

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



TEŞEKKÜRLER